

DEVOPS PORTFOLIO HANDBOOK

BUILD | AUTOMATE | SECURE | DEPLOY | OPERATE | IMPROVE

20 PROJECTS. REAL SKILLS. PRODUCTION THINKING.



LINUX



Git



docker



kubernetes



Terraform



ANSIBLE



Jenkins



argo



GitHub



Prometheus



Grafana



sonarqube



trivy



HashiCorp
Vault



S3



IAM



CloudWatch



Route 53



MySQL



CODE



BUILD



TEST



SECURE



PACKAGE



DEPLOY



MONITOR

VERIQTA[®]



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

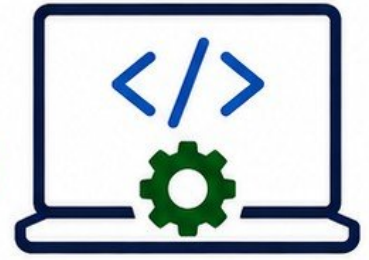


/VERIQTA



/VERIQTA

WHAT A STRONG DEVOPS PORTFOLIO PROVES



YOUR PORTFOLIO IS YOUR PROOF. MAKE IT SHOW YOU CAN ENGINEER, AUTOMATE, SECURE, OPERATE, AND IMPROVE REAL SYSTEMS.

1 WHAT EMPLOYERS SHOULD LEARN FROM YOUR PORTFOLIO



- Can you solve real problems?
- Can you design and build systems?
- Can you automate and ship code?
- Can you operate and monitor in production?
- Can you troubleshoot and recover from failure?
- Can you make secure, reliable, and scalable architecture decisions?

2 PROJECTS VERSUS TUTORIALS



PROJECTS

PROJECTS: Solve a real problem, make decisions, handle trade-offs, document why, and own the result.



TUTORIALS

TUTORIALS: Follow step-by-step instructions with no real decisions or ownership.

3 EVIDENCE OF ENGINEERING DECISIONS



- Why did you choose this architecture?
- Why this tool instead of another?
- What alternatives did you consider?
- What trade-offs did you accept?
- How does it handle failure?
- How does it scale?
- What did you optimize for and why?

4 SHOWING OWNERSHIP, NOT TOOL FAMILIARITY



- End-to-end responsibility
- Your ideas, your implementation
- Your documentation
- Your troubleshooting
- Your improvements
- Your lessons learned
- Your impact on the system

5 DEPTH VERSUS NUMBER OF REPOSITORIES



FEW GREAT PROJECTS > MANY BASIC PROJECTS

- Depth shows mastery
- Depth shows problem-solving
- Depth shows production thinking
- Depth shows real-world readiness

6 WHAT MAKES A PROJECT PORTFOLIO-WORTHY



- Solves a real or realistic problem
- Production-like architecture
- Automation and IaC
- Monitoring and observability
- Security considerations
- Failure handling and recovery
- Clear documentation and diagrams
- Shows measurable or visible outcomes

7 COMMON WEAK PORTFOLIO PATTERNS – AVOID THESE



ONLY TUTORIAL PROJECTS

No real decisions or ownership.



TOO MANY REPOS

Shallow work with no depth.



NO DOCUMENTATION OR DIAGRAMS

Hard to understand your thinking.



NO FAILURE TESTING OR RECOVERY

Doesn't prove you can handle real issues.



NO SECURITY CONSIDERATIONS

Ignores a critical part of DevOps.



NO EXPLANATION OF DECISIONS

Employers can't see your engineering.



PORTFOLIO ARCHITECTURE AND GITHUB STRUCTURE

A WELL-STRUCTURED PORTFOLIO IS EASY TO EXPLORE, EASY TO TRUST, AND DEMONSTRATES PROFESSIONAL ENGINEERING PRACTICE.

1 DESIGNING THE OVERALL PORTFOLIO



- Tell a story with your projects.
- Show progression of skills.
- Start with a strong "hero" project.
- Cover the DevOps lifecycle: Build, Automate, Secure, Deploy, Operate, Improve.
- Quality > Quantity.

2 REPOSITORY ORGANIZATION



- One project per repository.
- Use a clear and consistent structure.
- Keep code, docs, and configs well organized.
- Separate IaC, scripts, and diagrams.

3 NAMING CONVENTIONS



- Use clear, descriptive, and searchable names.
- Prefer: <domain>-<purpose>-<tech>
- Examples:
 - ecom-app-cicd-pipeline
 - aws-multi-az-terraform
 - k8s-gitops-argocd
 - monitoring-stack-prom-grafana

4 PINNED REPOSITORIES



- Pin your best 6 repositories.
- Lead with your strongest project.
- Show variety across the DevOps lifecycle.
- Keep pinned repos up-to-date and well documented.

5 GITHUB PROFILE README



- Create a custom profile README.
- Who you are and what you build.
- Your skills and focus areas.
- Highlight top projects.
- Add architecture snapshots.
- Add links to blog, portfolio site, and resume.

6 PROJECT CATEGORIES



- Organize projects across key areas:
 - ✓ Build & Code
 - ✓ Automation & CI/CD
 - ✓ Infrastructure as Code
 - ✓ Deployment & Orchestration
 - ✓ Monitoring & Observability
 - ✓ Security & Compliance
 - ✓ Reliability & Operations

7 DOCUMENTATION STANDARDS



- Every repo must have a strong README.
- Include: problem, architecture, setup, usage, decisions, and lessons.
- Use diagrams to explain, not to decorate.
- Keep documentation clean, short, and scannable.

8 KEEPING REPOSITORIES PROFESSIONAL



- Clean commit history.
- Consistent formatting.
- No hardcoded secrets.
- Meaningful commit messages.
- Use .gitignore properly.
- Remove clutter and test files.
- Review before you publish.

9 PUBLIC VERSUS PRIVATE INFORMATION



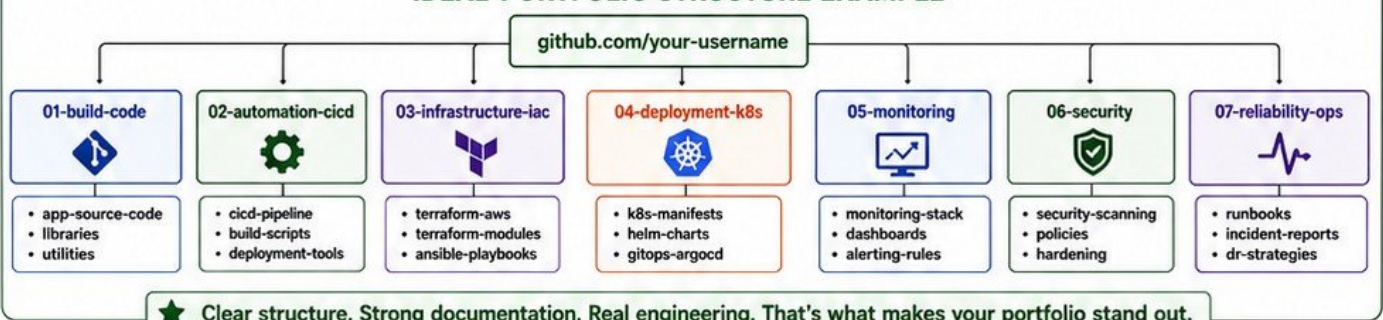
MAKE PUBLIC:

- Code (safe)
- Architecture diagrams
- Documentation
- Configs without secrets

KEEP PRIVATE:

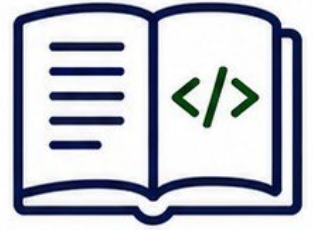
- Secrets and credentials
- Customer or company data
- Internal private scripts
- Sensitive infrastructure details

IDEAL PORTFOLIO STRUCTURE EXAMPLE



★ Clear structure. Strong documentation. Real engineering. That's what makes your portfolio stand out.

BUILD A PRODUCTION-GRADE PROJECT README



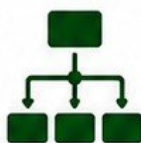
A GREAT README EXPLAINS YOUR PROJECT CLEARLY, SHOWS YOUR ENGINEERING THINKING, AND BUILDS TRUST.

1 PROBLEM STATEMENT



- What problem does this project solve?
- Who is the target user?
- Why is this problem important?
- What is the desired outcome?

2 ARCHITECTURE OVERVIEW



- High-level description of the solution.
- Key components and how they work together.
- Data flow overview.

3 TECHNOLOGY CHOICES



- List the main technologies and tools used.
- Why you chose each one.
- Alternatives considered and why not selected.

4 PREREQUISITES



- Required accounts (AWS, Docker Hub, etc.).
- Tools and versions needed.
- System requirements.
- Access and permissions.

5 DEPLOYMENT INSTRUCTIONS



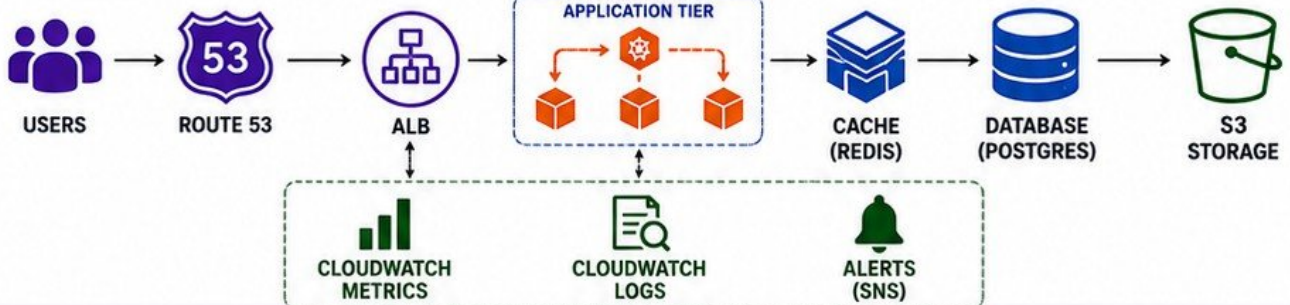
- Step-by-step setup.
- Infrastructure provisioning.
- Application deployment.
- Configuration steps.
- Verification steps.
- Cleanup (destroy) instructions.

6 ARCHITECTURE DIAGRAM



- Clear diagram of the system.
- Show components, connections, and data flow.
- Keep it simple but complete.

EXAMPLE ARCHITECTURE DIAGRAM



7 SECURITY CONSIDERATIONS



- Authentication & authorization.
- Network security (VPC, SG, NACL).
- Encryption (in transit & at rest).
- Secrets management.
- Least privilege access.

8 OBSERVABILITY



- Metrics to monitor.
- Logs and log retention.
- Dashboards and alerts.
- Health checks.
- Tracing (if applicable).

9 FAILURE SCENARIOS



- What can go wrong?
- How the system handles failure.
- Recovery and rollback mechanisms.
- Manual intervention steps.

10 TRADE-OFFS



- What trade-offs did you make?
- Why did you accept them?
- Cost vs performance.
- Simplicity vs flexibility.
- Managed vs self-managed.

11 KNOWN LIMITATIONS



- What is not solved yet?
- What are the constraints?
- Which features are out of scope?
- What could be improved?

12 LESSONS LEARNED



- What did you learn?
- What would you do differently?
- Key takeaways.
- Next improvements.

★ A GREAT README IS YOUR PROJECT'S FIRST IMPRESSION. MAKE IT CLEAR, COMPLETE, AND PROFESSIONAL.



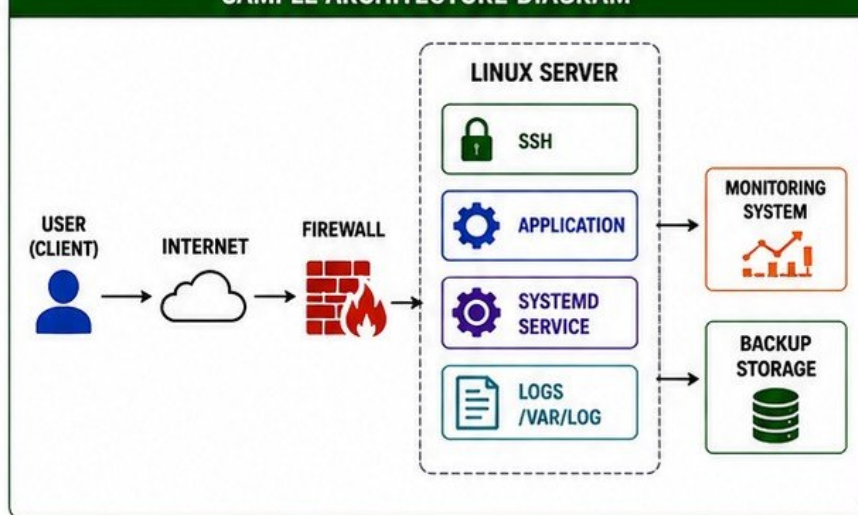
LINUX PRODUCTION OPERATIONS PROJECT

BUILD AND DOCUMENT A LINUX-HOSTED APPLICATION ENVIRONMENT THAT IS SECURE, RELIABLE, AND PRODUCTION-READY.

INCLUDE THESE ELEMENTS

- LINUX SERVER PROVISIONING**
Provision a clean Linux server (cloud or on-prem). Document OS, version, and initial setup.
- USERS AND GROUPS**
Create non-root users, groups, and sudo access with least privilege.
- SSH HARDENING**
Disable root login, use key-based auth, change default port, configure allow users, and SSH settings.
- FILE PERMISSIONS**
Set correct ownership and permissions. Follow least privilege for files and directories.
- SYSTEMD SERVICE**
Create a systemd service for your application. Enable auto-start, restart, and proper dependencies.
- PROCESS MANAGEMENT**
Use systemctl, ps, top, htop to monitor and manage processes.
- LOG MANAGEMENT**
Centralize logs to /var/log or journald. Rotate logs and keep disk usage in check.
- FIREWALL CONFIGURATION**
Configure ufw/iptables/firewalld. Allow only required ports. Deny everything else.
- RESOURCE MONITORING**
Monitor CPU, memory, disk, and network. Use tools like df, free, vmstat, iostat, and sar.
- TROUBLESHOOTING EVIDENCE**
Document real issues, how you diagnosed them, and the fix. Show commands and outputs.

SAMPLE ARCHITECTURE DIAGRAM



PORTFOLIO EVIDENCE – SHOW THESE

CONFIGURATION FILES	COMMANDS USED	ARCHITECTURE DIAGRAM	FAILURE INVESTIGATION	SECURITY DECISIONS
/etc/ssh/sshd_config /etc/sudoers.d/* - systemd service file - ufw/firewall rules - application config	- Server setup - User & group management - SSH hardening - Service management - Monitoring - Firewall setup	- Clear view of components - Data flow - Security layers - External dependencies	- Issue description - Symptoms - Root cause - Investigation steps - Fix applied - Prevention	- Why these users and permissions - SSH and firewall choices - Least privilege approach - Risk reduction strategy

PROJECT GOALS



- ✓ Deploy a secure and stable Linux environment.
- ✓ Run an application as a service.
- ✓ Ensure availability, performance, and security.
- ✓ Document every step and decision.
- ✓ Be able to troubleshoot and recover.

SUCCESS CRITERIA



- ✓ Server is secure and hardened.
- ✓ Application runs as a systemd service.
- ✓ Logs are managed and monitored.
- ✓ System resources are monitored.
- ✓ Issues are investigated and resolved.
- ✓ Documentation is complete and clear.

GIT AND COLLABORATIVE DEVELOPMENT PROJECT

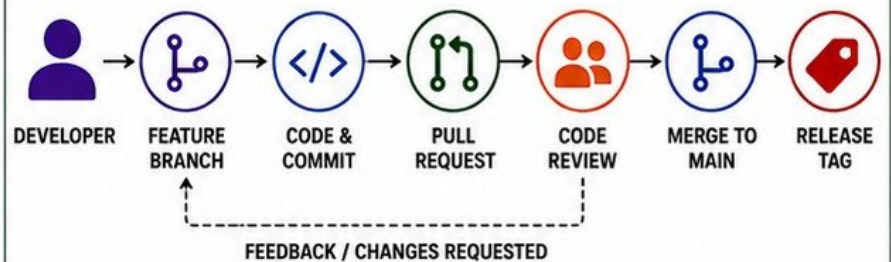


DEMONSTRATE A PROFESSIONAL GIT WORKFLOW
THAT SUPPORTS TEAMWORK, QUALITY, AND SAFE DELIVERY.

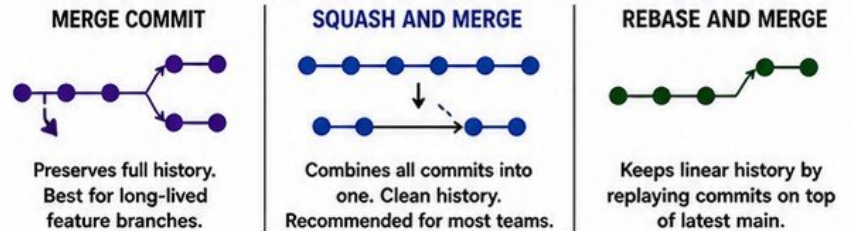
INCLUDE THESE ELEMENTS

- 1** **FEATURE BRANCHES**
Create a new branch for every feature, fix, or improvement. Keep main clean and stable.
- 2** **PULL REQUESTS**
Open a PR to propose changes. Provide context, screenshots, and testing notes.
- 3** **PROTECTED BRANCHES**
Protect main and release branches. Require PRs, reviews, and status checks before merging.
- 4** **CODE REVIEWS**
At least one approval required. Focus on code quality, security, and best practices.
- 5** **MERGE STRATEGIES**
Use the right strategy: Merge Commit, Squash and Merge, or Rebase and Merge.
- 6** **TAGS AND RELEASES**
Tag stable versions. Create GitHub Releases with changelogs and artifacts.
- 7** **CONVENTIONAL COMMITS**
Follow a standard format for clear history and automation. Example: feat: add user login
- 8** **CONFLICT RESOLUTION**
Resolve conflicts carefully. Test changes before committing and pushing.
- 9** **ROLLBACK STRATEGY**
Revert commits or releases when issues occur. Keep history clean and traceable.
- 10** **GITHUB REPOSITORY POLICIES**
Define branch protection rules, code review requirements, status checks, and signing.

PROFESSIONAL GIT WORKFLOW



MERGE STRATEGIES EXPLAINED

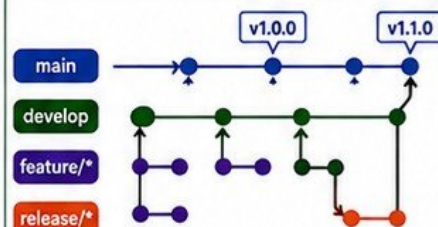


CONVENTIONAL COMMIT FORMAT (EXAMPLE)

`<type>(<scope>): <subject>`
[optional body]
[optional footer]

TYPES: feat, fix, docs, style, refactor, test, chore, ci, build, perf, revert
EXAMPLE: feat(auth): add JWT login with refresh token support

BRANCHING MODEL (EXAMPLE)



BEST PRACTICES

- ✓ Commit small and focused changes.
- ✓ Write clear commit messages.
- ✓ Keep branches updated.
- ✓ Run tests before pushing.
- ✓ Review before merging.
- ✓ Delete merged branches.
- ✓ Protect main and release branches.
- ✓ Automate checks in CI.
- ✓ Document contribution guidelines.

★ A STRONG GIT WORKFLOW BUILDS TRUST, IMPROVES CODE QUALITY, AND ENSURES SAFE, COLLABORATIVE DELIVERY.



DOCKERIZED APPLICATION PROJECT

CONTAINERIZE A REALISTIC APPLICATION AND DESIGN IT FOR PRODUCTION.
NOT JUST A CONTAINER THAT RUNS, BUT ONE THAT IS SECURE, RELIABLE, AND EFFICIENT.

INCLUDE THESE ELEMENTS

- 1 **DOCKERFILE**
Write a clear and efficient Dockerfile that builds and runs the application consistently.
- 2 **MULTI-STAGE BUILDS**
Use multi-stage builds to reduce the final image size and improve security.
- 3 **.DOCKERIGNORE**
Exclude files and directories that are not needed in the build context.
- 4 **ENVIRONMENT CONFIGURATION**
Use environment variables and configuration files. Avoid hardcoding values.
- 5 **NON-ROOT CONTAINERS**
Run the container as a non-root user to improve security.
- 6 **HEALTH CHECKS**
Add health checks so Docker and orchestrators can detect unhealthy containers.
- 7 **IMAGE TAGGING**
Use meaningful and consistent tags. Example: app:1.0.0, app:latest, app:<git-sha>.
- 8 **CONTAINER NETWORKING**
Expose only required ports. Use user-defined networks for communication.
- 9 **PERSISTENT DATA**
Use volumes or bind mounts for persistent data. Never store data inside the image.
- 10 **IMAGE VULNERABILITY SCANNING**
Scan images for known vulnerabilities using tools like Trivy or Gypre.
- 11 **IMAGE-SIZE REDUCTION**
Remove unnecessary packages, clean caches, use slim base images, and multi-stage builds.

SAMPLE DOCKERFILE (MULTI-STAGE)

```
# ----- BUILD STAGE -----
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ----- PRODUCTION STAGE -----
FROM node:20-alpine AS production
WORKDIR /app
ENV NODE_ENV=production
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=builder /app/dist ./dist
COPY package*.json ./
RUN npm ci --omit=dev \
  && chown -R appuser:appgroup /app

USER appuser
EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -q -O - http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

BEST PRACTICES CHECKLIST

- ✓ Minimal base image
- ✓ Multi-stage build
- ✓ Non-root user
- ✓ Only necessary packages
- ✓ Environment variables
- ✓ Health checks
- ✓ .dockerignore used
- ✓ No secrets in image
- ✓ Proper tagging
- ✓ Read-only filesystem (optional)
- ✓ Drop unnecessary capabilities
- ✓ Resource limits (in orchestrator)
- ✓ Documentation in README

CONTAINER THAT RUNS (NOT PRODUCTION READY)



CONTAINER DESIGNED FOR PRODUCTION



node:latest	BASE IMAGE	node:20-alpine (specific, minimal)
Runs as root	USER	Runs as non-root user
1.2 GB	IMAGE SIZE	120 MB
No health check	HEALTH CHECK	Health check configured
Hardcoded in image	SECRETS	Environment variables / secrets manager
All files copied	FILES INCLUDED	.dockerignore used
No vulnerability scan	SECURITY	Scanned for vulnerabilities
Data inside container	PERSISTENCE	Volumes for persistent data
All ports exposed	NETWORKING	Only required ports exposed
Fails silently	RELIABILITY	Health check + restart policy

PORTFOLIO EVIDENCE – SHOW THESE



DOCKERFILE
(EXPLAINED)



BUILD & RUN
COMMANDS



IMAGE SIZE
COMPARISON



VULNERABILITY
SCAN REPORT



NETWORK
DIAGRAM



VOLUME
CONFIGURATION



HEALTH CHECK
TEST RESULTS

★ BUILD CONTAINERS THAT ARE SECURE, RELIABLE, MAINTAINABLE, AND READY FOR PRODUCTION.

CI/CD PIPELINE PROJECT



BUILD A COMPLETE AUTOMATED DELIVERY PIPELINE THAT IS RELIABLE, SECURE, AND READY FOR PRODUCTION.



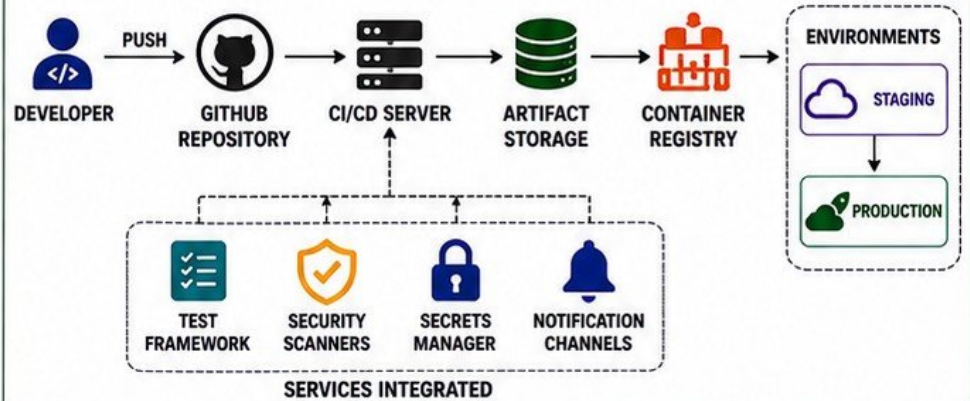
PIPELINE FLOW



INCLUDE THESE ELEMENTS

- PIPELINE AS CODE**
Define the entire pipeline in code using YAML (GitHub Actions, GitLab CI, Jenkins, etc.).
- AUTOMATED TESTS**
Run unit, integration, and end-to-end tests automatically. Fail fast on any issue.
- ARTIFACT MANAGEMENT**
Store and version build artifacts (binaries, packages, reports) for traceability and reuse.
- SECRETS MANAGEMENT**
Use secure secrets storage (Vault, AWS Secrets Manager, GitHub Secrets, etc.).
- APPROVAL GATES**
Require manual approval before deploying to sensitive environments (staging/production).
- DEPLOYMENT VERIFICATION**
Verify deployments with smoke tests, health checks, and automated validation.
- FAILURE HANDLING**
Fail the pipeline on errors. Provide clear logs and diagnostics.
- NOTIFICATIONS**
Send notifications to Slack, Teams, email, or PagerDuty at each stage.
- ROLLBACK STRATEGY**
Automate rollback to the last known good release on failure or manual trigger.
- AUDIT & TRACEABILITY**
Maintain logs, build history, deployment records, and change traceability.

SAMPLE PIPELINE ARCHITECTURE



EXAMPLE PIPELINE (YAML SNIPPET)

```
stages:
  - validate
  - build
  - test
  - security-scan
  - build-image
  - push-image
  - deploy-staging
  - verify
  - deploy-production

validate:
  script: npm run lint

build:
  script: npm run build

test:
  script: npm test

security-scan:
  script: trivy fs .

build-image:
  script: docker build -t myapp:$CI_COMMIT_SHA .

push-image:
  script: docker push registry.example.com/myapp:$CI_COMMIT_SHA

deploy-staging:
  script: kubectl apply -f k8s/staging/

verify:
  script: ./scripts/smoke-test.sh

deploy-production:
  when: manual
  script: kubectl apply -f k8s/production/

environment:
  name: production
  url: https://myapp.example.com
```

KEY SUCCESS FACTORS

- ✓ Fast feedback and early failure.
- ✓ Automated quality, security, and tests.
- ✓ Reproducible builds and deployments.
- ✓ Secure secrets and least-privilege access.
- ✓ Reliable rollback and recovery.
- ✓ Visibility, notifications, and auditability.

PIPELINE QUALITY CHECKLIST

- ✓ Pipeline as code versioned
- ✓ Tests automated and reliable
- ✓ Security scans integrated
- ✓ Artifacts stored and traceable
- ✓ Secrets managed securely
- ✓ Approval gates in place
- ✓ Deployments verified
- ✓ Failures handled gracefully
- ✓ Notifications configured
- ✓ Rollback tested and documented



AUTOMATE EVERYTHING. VERIFY EVERY DEPLOYMENT. DELIVER WITH CONFIDENCE.





Terraform

TERRAFORM INFRASTRUCTURE PROJECT

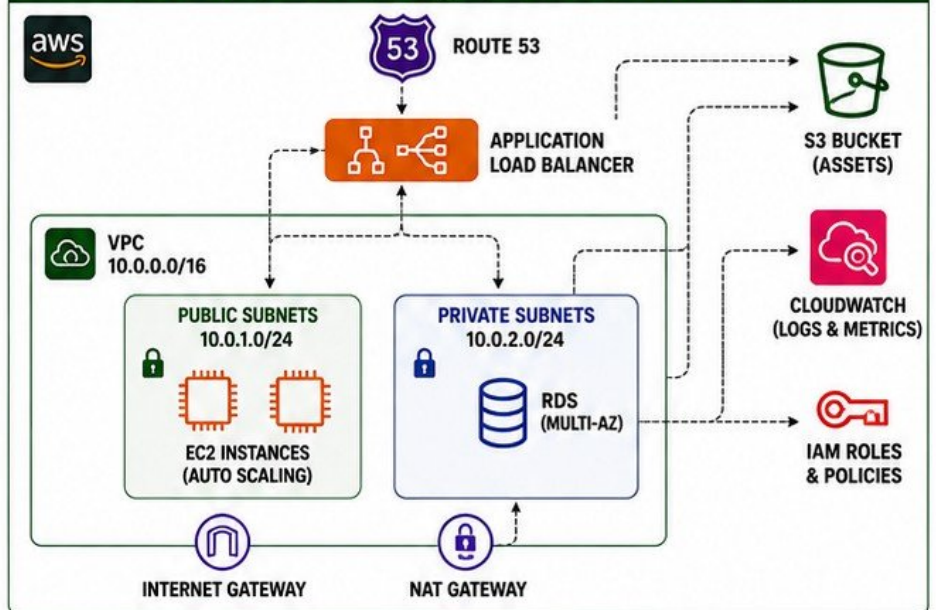
PROVISION CLOUD INFRASTRUCTURE USING TERRAFORM.

INFRASTRUCTURE AS CODE THAT IS REUSABLE, SCALABLE, AND AUDITABLE.

INCLUDE THESE ELEMENTS

- 1 REMOTE STATE**
Store state in remote backend (S3, GCS, Azure Blob, etc.) for team collaboration.
- 2 STATE LOCKING**
Prevent concurrent changes using state locking (DynamoDB, Azure Table, etc.).
- 3 MODULES**
Organize reusable infrastructure components with modules.
- 4 VARIABLES**
Use input variables for flexibility and reusability across environments.
- 5 OUTPUTS**
Expose important values (IDs, endpoints, ARNs) as outputs.
- 6 PROVIDER CONSTRAINTS**
Pin provider versions to ensure consistency and avoid breaking changes.
- 7 ENVIRONMENT SEPARATION**
Separate environments (dev, staging, prod) using workspaces or directory structure.
- 8 IAM**
Create least-privilege IAM roles and policies for secure access.
- 9 NETWORKING**
Design VPC, subnets, route tables, IGW, NAT, security groups, and NACLs.
- 10 COMPUTE**
Provision compute resources (EC2, ASG, Lambda, etc.) as code.
- 11 STORAGE**
Manage storage resources (S3, EBS, RDS, EFS, etc.) securely.
- 12 TERRAFORM PLAN IN CI**
Run "terraform plan" in CI to validate changes before apply.
- 13 DRIFT CONSIDERATIONS**
Detect and handle configuration drift. Use "terraform plan" regularly.

SAMPLE AWS ARCHITECTURE (PROVISIONED WITH TERRAFORM)



EXAMPLE TERRAFORM CODE STRUCTURE

```

terraform-infra/
├── envs/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   ├── staging/
│   │   ├── ...
│   └── prod/
│       ├── ...
├── modules/
│   ├── vpc/
│   ├── ec2/
│   ├── rds/
│   └── s3/
├── backend.tf
├── providers.tf
└── README.md
  
```

EXAMPLE: MAIN.TF (SNIPPET)

```

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket     = "my-terraform-state"
  key        = "prod/terraform.tfstate"
  region     = "us-east-1"
  dynamodb_table = "terraform-locks"
  encrypt    = true
}

provider "aws" {
  region = var.aws_region
}
  
```

SAMPLE VARIABLES

```

variable "aws_region" {
  description = "AWS Region"
  type        = string
}

variable "vpc_cidr" {
  type        = string
  default    = "10.0.0.0/16"
}

variable "instance_type" {
  type        = string
  default    = "t3.medium"
}
  
```

PORTFOLIO EVIDENCE - SHOW THESE

TERRAFORM
CODEPLAN
OUTPUTREMOTE STATE
CONFIGSTATE LOCKING
SETUPARCHITECTURE
DIAGRAMDECISIONS &
TRADE-OFFS

BEST PRACTICES

- ✓ Use remote state with locking.
- ✓ Keep modules small, reusable, and versioned.
- ✓ Validate with 'terraform fmt', 'validate', and 'tftint'.
- ✓ Review plans before applying.
- ✓ Use least-privilege IAM.
- ✓ Separate environments cleanly.
- ✓ Document design decisions and trade-offs.
- ✓ Monitor for drift and reconcile.

★ INFRASTRUCTURE AS CODE ENABLES CONSISTENCY, REPEATABILITY,
AND SAFE COLLABORATION AT SCALE.

AWS PRODUCTION ARCHITECTURE PROJECT



DESIGN A HIGHLY AVAILABLE, SECURE, SCALABLE, AND COST-EFFECTIVE AWS WORKLOAD FOR REAL-WORLD PRODUCTION USE.

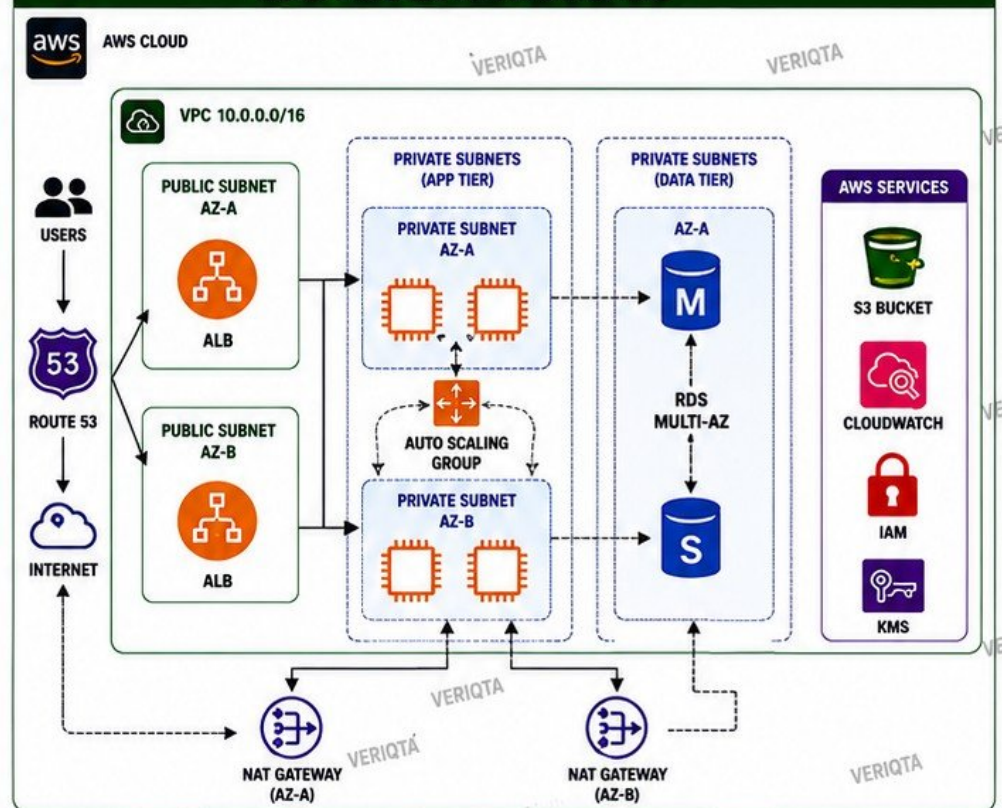
EXAMPLE ARCHITECTURE: ROUTE 53 → ALB → MULTI-AZ APPLICATION TIER → DATABASE



INCLUDE THESE ELEMENTS

- VPC**
Isolated network with CIDR blocks, subnets, route tables, and gateways.
- PUBLIC & PRIVATE SUBNETS**
Public subnets for ALB, NAT. Private subnets for application and database tiers.
- ROUTE TABLES**
Custom route tables for public and private subnets with correct routes.
- NAT GATEWAY**
Enable outbound internet access for private subnets without exposing resources.
- SECURITY GROUPS**
Least-privilege access controls between all components.
- LOAD BALANCING**
Application Load Balancer across multiple AZs for high availability.
- AUTO SCALING**
Scale application instances based on demand and health.
- RDS DATABASE**
Multi-AZ database with automatic backups and failover.
- S3 BUCKET**
Store static assets, backups, logs, and application data.
- IAM**
Roles, policies, and least-privilege access for all services and users.
- CLOUDWATCH**
Monitoring, alarms, logs, dashboards, and metrics for observability.

HIGHLY AVAILABLE AWS ARCHITECTURE



AVAILABILITY

- Multi-AZ ALB, Application Tier, and RDS.
- Auto Scaling maintains capacity during failures.
- Route 53 health checks for fault tolerance.
- S3 and RDS backups for disaster recovery.

SECURITY

- Private subnets for application and database.
- Security groups restrict traffic at all layers.
- IAM least-privilege policies.
- Encrypted data in transit and at rest (KMS).

SCALING

- Auto Scaling adds/removes instances automatically.
- ALB distributes traffic evenly.
- RDS read replicas for read scaling (optional).
- S3 scales automatically.

COST TRADE-OFFS

- NAT Gateways incur hourly costs.
- Multi-AZ RDS increases cost for higher availability.
- Right-sizing instances saves costs.
- Use S3 lifecycle policies and CloudWatch alarms to optimize.

PORTFOLIO EVIDENCE – SHOW THESE



★ DESIGN FOR RESILIENCE. SECURE BY DEFAULT. SCALE SMART. OPTIMIZE COSTS. ★



ANSIBLE

ANSIBLE CONFIGURATION MANAGEMENT PROJECT

AUTOMATE SERVER CONFIGURATION. CONSISTENT. REPEATABLE. RELIABLE.

ANSIBLE ENSURES SERVERS ARE CONFIGURED THE SAME WAY EVERY TIME.

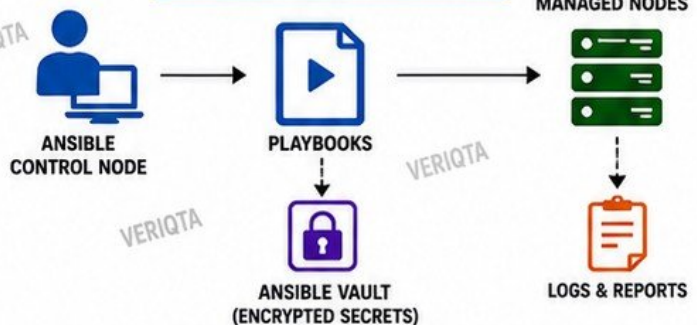
WHAT THIS PROJECT DEMONSTRATES

- ✓ Automated server setup and configuration
- ✓ Consistency across environments
- ✓ Secure secrets management
- ✓ Idempotent and reliable automation
- ✓ Faster deployments and updates
- ✓ Reduced manual work and human error
- ✓ Production-ready operational practices



ANSIBLE

AUTOMATION ARCHITECTURE



KEY ANSIBLE CONCEPTS USED

- | | | |
|----|--------------------------|--|
| 1 | INVENTORY | Define and group hosts statically or dynamically. |
| 2 | ROLES | Organize tasks, variables, handlers, templates and files. |
| 3 | VARIABLES | Store configuration values for different environments. |
| 4 | TEMPLATES | Use Jinja2 templates for dynamic configuration files. |
| 5 | HANDLERS | Run tasks only when notified by changes. |
| 6 | VAULT | Encrypt sensitive data like passwords and keys. |
| 7 | PRIVILEGE ESCALATION | Use become to perform tasks as another user (sudo). |
| 8 | IDEMPOTENCY | Run playbooks multiple times without causing changes. |
| 9 | APPLICATION INSTALLATION | Install packages, copy files, and set correct permissions. |
| 10 | CONFIGURATION DEPLOYMENT | Deploy configs using templates, files, and modules. |
| 11 | SERVICE MANAGEMENT | Start, stop, enable, restart and manage services. |
| 12 | FAILURE HANDLING | Use error handling, retries, and rescue blocks. |

EXAMPLE PLAYBOOK (SITE.YML)

```

---
- name: Configure and deploy application
  hosts: webserver
  become: yes
  vars_files:
    - vars/main.yml
    - vault/secrets.yml
  roles:
    - common
    - application
    - monitoring
  handlers:
    - import_tasks: handlers/main.yml
  
```

EXAMPLE DIRECTORY STRUCTURE

```

ansible-project/
├── inventory/
│   ├── hosts.yml
│   └── group_vars/
│       └── all.yml
├── roles/
│   ├── common/
│   ├── application/
│   └── monitoring/
├── playbooks/
│   └── site.yml
├── vars/
│   └── main.yml
├── vault/
│   └── secrets.yml
└── README.md
  
```

IDEMPOTENCY PROOF

RUN PLAYBOOK MULTIPLE TIMES - SAME RESULT

```

$ ansible-playbook -i inventory/hosts.yml playbooks/site.yml
PLAY RECAP *****
web1      : ok=20  changed=15  unreachable=0  failed=0
web2      : ok=20  changed=15  unreachable=0  failed=0

$ ansible-playbook -i inventory/hosts.yml playbooks/site.yml
PLAY RECAP *****
web1      : ok=20  changed=0   unreachable=0  failed=0
web2      : ok=20  changed=0   unreachable=0  failed=0

NO CHANGES ON SECOND RUN = IDEMPOTENT!
  
```

EXAMPLE COMMANDS

```

ansible all -m ping                # Test connectivity
ansible-playbook -i inventory/hosts.yml site.yml # Run playbook
ansible all -m setup                # Gather facts
ansible-vault create vault/secrets.yml # Create encrypted file
ansible-playbook --check site.yml    # Dry run (check mode)
ansible-playbook --diff site.yml     # Show changes preview
  
```

BEST PRACTICES

- ✓ Use roles for reusability
- ✓ Keep variables organized
- ✓ Encrypt sensitive data
- ✓ Use least privilege (become)
- ✓ Test with --check and --diff
- ✓ Write idempotent tasks
- ✓ Handle failures gracefully
- ✓ Document everything
- ✓ Version control your code
- ✓ Monitor and review logs



ANSIBLE MAKES INFRASTRUCTURE CONSISTENT, SECURE AND EASY TO MANAGE.
RUN IT ONCE OR A HUNDRED TIMES - THE RESULT IS PREDICTABLE.

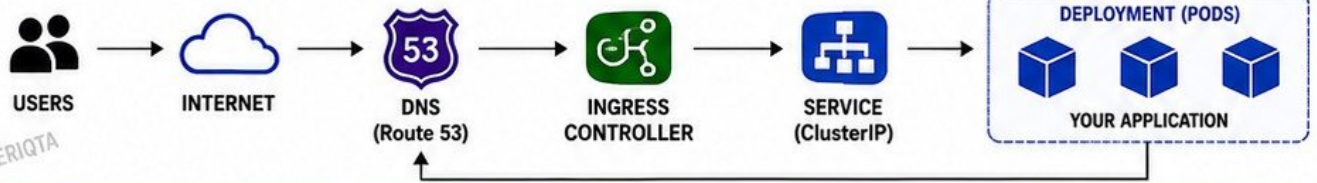


KUBERNETES APPLICATION DEPLOYMENT PROJECT



DEPLOY A SCALABLE, SECURE, AND PRODUCTION-READY APPLICATION ON KUBERNETES.

APPLICATION ARCHITECTURE – HOW TRAFFIC REACHES THE WORKLOAD



KEY KUBERNETES COMPONENTS USED

- DEPLOYMENTS**
Manages Pods and ensures the desired number of replicas are running.
- SERVICES**
Provides stable networking and load balancing to Pods.
- CONFIGMAPS**
Stores non-sensitive configuration data as key-value pairs.
- SECRETS**
Stores sensitive data like passwords, tokens, and keys.
- NAMESPACES**
Isolates and organizes resources within the cluster.
- RESOURCE REQUESTS & LIMITS**
Ensures Pods get the right resources and prevents resource exhaustion.
- LIVENESS PROBES**
Detects if the container is alive. Restarts it if it fails.
- READINESS PROBES**
Detects if the container is ready to receive traffic.
- INGRESS**
Exposes HTTP/HTTPS routes from outside the cluster.
- PERSISTENT STORAGE**
Stores data outside the Pod lifecycle.
- ROLLING UPDATES**
Updates the application with zero downtime.
- ROLLBACKS**
Roll back to a previous stable version quickly and safely.

EXAMPLE MANIFESTS (YAML)

DEPLOYMENT

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginxdemos/hello
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "256Mi"
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 10
        periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 5
  
```

SERVICE

```

apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  
```

CONFIGMAP

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_NAME: "MyApp"
  APP_ENV: "production"
  LOG_LEVEL: "info"
  
```

SECRET

```

apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  username: YWRtaW4=
  password: cGZlc3RvcnQ=
  
```

INGRESS

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80
  
```

PERSISTENT VOLUME CLAIM

```

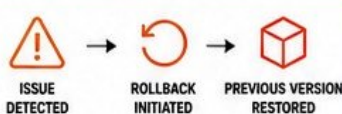
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  
```

ROLLING UPDATE FLOW



ZERO DOWNTIME DEPLOYMENT

ROLLBACK FLOW



QUICK RECOVERY TO STABLE STATE

WHAT THIS PROJECT PROVES

- ✓ I can deploy real applications on Kubernetes.
- ✓ I understand core Kubernetes objects.
- ✓ I can configure apps for reliability.
- ✓ I can expose apps securely.
- ✓ I can manage updates and rollbacks.
- ✓ I can persist data safely.
- ✓ I can design for production.



A WELL-ARCHITECTED KUBERNETES DEPLOYMENT = RELIABLE • SCALABLE • MAINTAINABLE • PRODUCTION READY

PRODUCTION KUBERNETES PROJECT



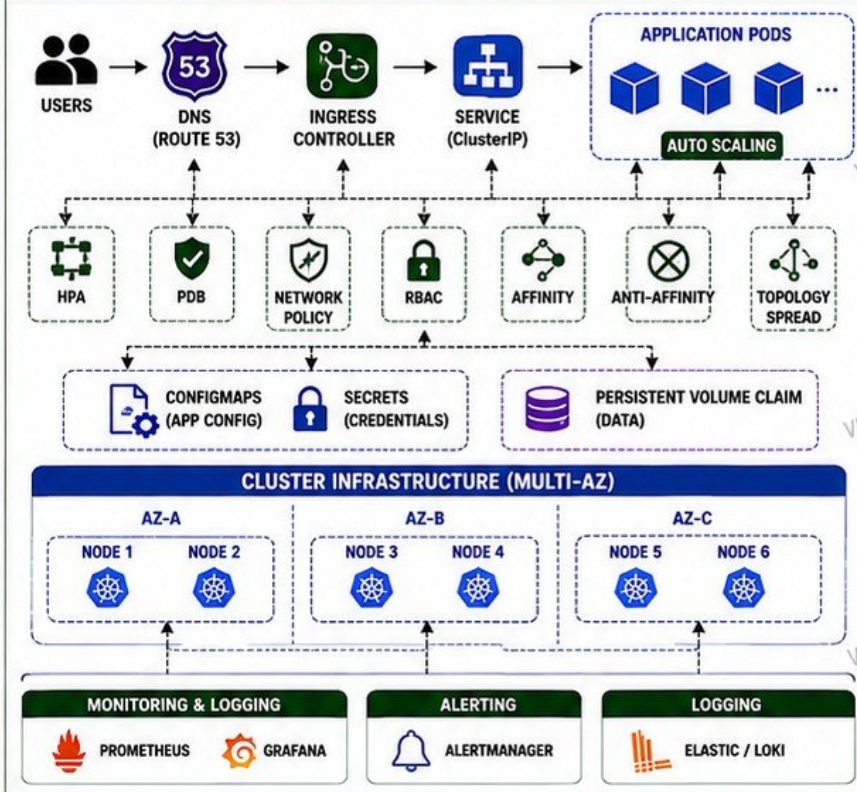
kubernetes

MOVE BEYOND A BASIC DEPLOYMENT.
BUILD RESILIENT, SECURE, AND HIGHLY AVAILABLE APPLICATIONS.

PRODUCTION KUBERNETES FEATURES

- HORIZONTAL POD AUTOSCALER**
Automatically scales pods based on CPU, memory, or custom metrics.
- POD DISRUPTION BUDGET**
Ensures minimum availability during voluntary disruptions.
- NETWORK POLICY**
Controls traffic between pods and namespaces.
- RBAC**
Restricts access using roles, rolebindings, and service accounts.
- AFFINITY**
Places pods on preferred nodes or zones.
- ANTI-AFFINITY**
Prevents pods from running together on the same node.
- TOPOLOGY SPREAD**
Distributes pods evenly across zones and nodes.
- SECURITY CONTEXTS**
Runs containers with least privilege and hardened settings.
- GRACEFUL TERMINATION**
Allows pods to shutdown cleanly before termination.
- HIGH AVAILABILITY**
Multi-AZ, self-healing, and redundant by design.
- FAILURE RECOVERY**
Detects failures and automatically recovers workloads.
- PRODUCTION OBSERVABILITY**
Monitor health, performance, and capacity at all times.

PRODUCTION KUBERNETES ARCHITECTURE



WHAT HAPPENS WHEN THINGS FAIL?

POD FAILURE	NODE FAILURE	DEPENDENCY FAILURE	ZONE FAILURE	CONFIG CHANGE ERROR	OUTCOME
- Liveness probe fails - Pod restarted - If still failing, new pod scheduled	- Node becomes NotReady - Pods rescheduled on healthy nodes - Service continues	- Connection errors occur - Readiness probe fails - Traffic stopped to pod - Alert triggered	- AZ goes down - Traffic rerouted to healthy AZs - Capacity withstands load	- Bad config deployed - Rollback triggered - Previous good state restored	THE SYSTEM DETECTS, RECOVERS AND STAYS AVAILABLE.
RESULT: SELF-HEALED	RESULT: WORKLOAD RECOVERED	RESULT: IMPACT MINIMIZED	RESULT: HIGH AVAILABILITY	RESULT: ROLLBACK SUCCESS	

BEST PRACTICES

- Design for failure, not just for success.
- Keep replicas across multiple zones.
- Use health checks for everything.
- Enforce least privilege with RBAC.
- Limit blast radius with NetworkPolicies.
- Automate scaling for changing demand.
- Test failure scenarios regularly.
- Observe everything. Alert early.

EXAMPLE MANIFEST SNIPPETS

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: webapp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: webapp
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  
```

```

apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: webapp-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: webapp
  
```

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: webapp-network-policy
spec:
  podSelector:
    matchLabels:
      app: webapp
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector: {}
    ports:
    - protocol: TCP
      port: 80
  
```

PRODUCTION READINESS CHECKLIST

- HPA configured and tested
- PDB protects workloads
- NetworkPolicies restrict traffic
- RBAC follows least privilege
- Pods spread across zones
- Resource requests and limits set
- Liveness and readiness probes enabled
- Graceful termination configured
- Backups and DR strategy in place
- Monitoring and alerting active
- Failures tested and documented

★ BUILD RESILIENT SYSTEMS THAT HEAL THEMSELVES. FAILURES ARE INEVITABLE – DOWNTIME IS OPTIONAL. ★

GITOPS DEPLOYMENT PROJECT

BUILD A GIT-DRIVEN DEPLOYMENT WORKFLOW.
DESIRED STATE. AUTOMATED SYNCHRONIZATION. CONTINUOUS DELIVERY.



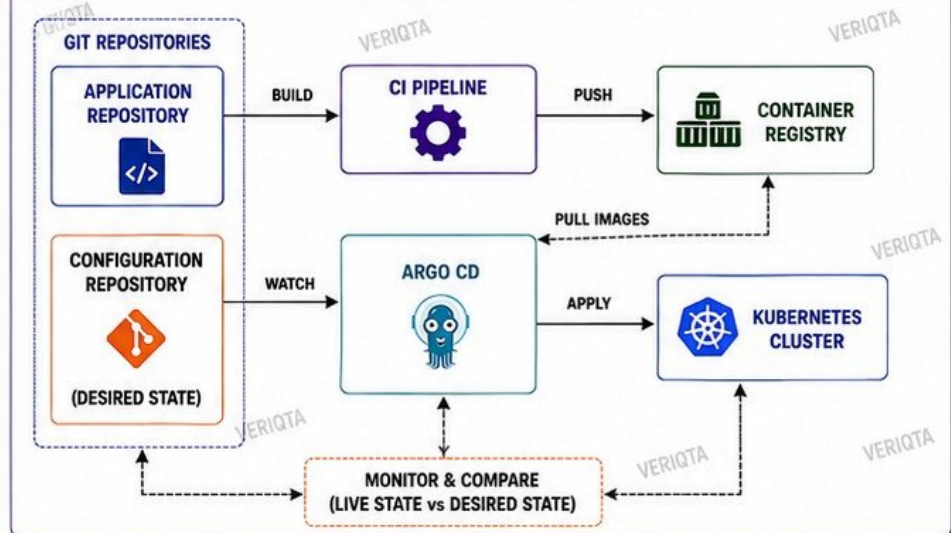
GITOPS WORKFLOW – END TO END



KEY COMPONENTS

- 1** **APPLICATION REPOSITORY**
Contains application source code and build configurations.
- 2** **CONFIGURATION REPOSITORY**
Stores Kubernetes manifests, Helm charts, Kustomize, and environment configurations.
- 3** **ARGO CD**
Continuously monitors Git and synchronizes the cluster to match the desired state.
- 4** **DESIRED STATE**
The state defined in Git. This is the single source of truth.
- 5** **AUTOMATED SYNCHRONIZATION**
Argo CD automatically applies changes to match the desired state.
- 6** **DRIFT DETECTION**
Detects any difference between live state and desired state.
- 7** **ENVIRONMENT PROMOTION**
Promote changes across environments using Git branches or folders.
- 8** **ROLLBACKS**
Revert to any previous Git commit to instantly roll back changes.
- 9** **GIT AUDIT TRAIL**
Every change is versioned, reviewed, and traceable.

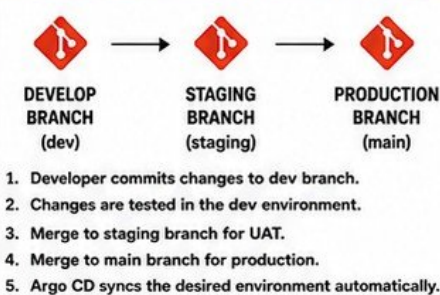
GITOPS ARCHITECTURE DIAGRAM



WHY GITOPS?

- | | | | | | |
|--|---|--|--|---|--|
| RELIABLE
Declarative, consistent and repeatable deployments. | SECURE
RBAC, least privilege, and audit-friendly workflows. | VISIBLE
Real-time visibility into application state. | AUTOMATED
Continuous delivery without manual intervention. | RECOVERABLE
Roll back instantly by reverting Git commits. | AUDITABLE
Full history of every change in Git. |
|--|---|--|--|---|--|

EXAMPLE ENVIRONMENT PROMOTION FLOW



EXAMPLE ARGO CD COMMANDS

```

# List applications
argocd app list

# Sync an application
argocd app sync my-app

# Check application status
argocd app get my-app

# Detect drift
argocd app diff my-app

# Rollback to a specific revision
argocd app rollback my-app --revision 1.2.3

# Refresh application
argocd app refresh my-app
  
```

BEST PRACTICES

- ✓ Store everything in Git.
- ✓ Use Pull Request reviews for all changes.
- ✓ Keep configurations environment-specific.
- ✓ Use RBAC for Argo CD.
- ✓ Enable automated sync with self-heal.
- ✓ Monitor and alert on drift.
- ✓ Test rollbacks regularly.
- ✓ Document application dependencies.
- ✓ Use tags for versioned releases.



GIT IS THE SOURCE OF TRUTH. ARGO CD IS THE ENFORCER. KUBERNETES IS THE EXECUTION ENGINE.



MONITORING AND OBSERVABILITY PROJECT



BUILD OBSERVABILITY AROUND YOUR APPLICATION.
SEE WHAT MATTERS. UNDERSTAND WHAT HAPPENS. RESPOND FASTER.

OBSERVABILITY PILLARS



METRICS

Quantitative measurements over time. Best for trends and alerting.



LOGS

Discrete events with context. Best for troubleshooting and forensics.



TRACES

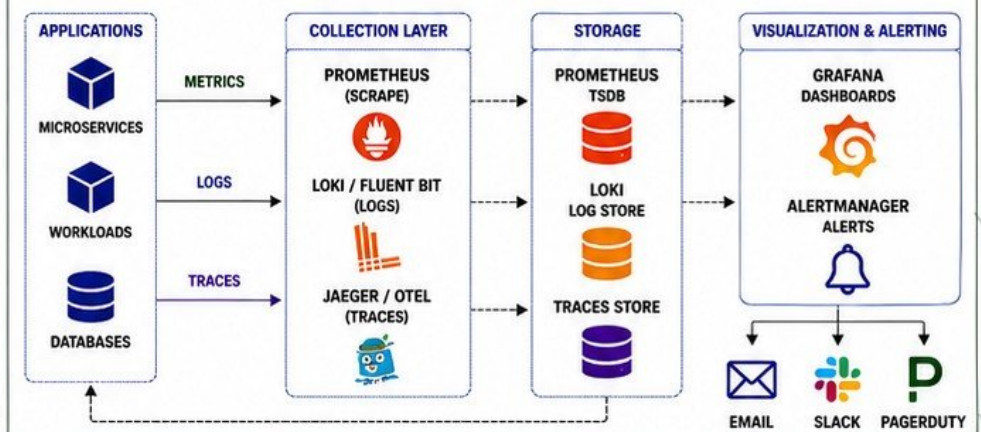
Request flow across services. Best for latency analysis and dependencies.



HEALTH

Simple signals that show if the system is alive and ready to serve.

OBSERVABILITY ARCHITECTURE



WHAT WE MONITOR (EXAMPLES)



APPLICATION METRICS

Request rate, error rate, latency, saturation, business KPIs.



INFRASTRUCTURE METRICS

CPU, memory, disk, network, node health, container health.



LOGS

Application logs, access logs, system logs, audit logs.



HEALTH INDICATORS

Readiness, liveness, dependency health, external service checks.

EXAMPLE DASHBOARDS (WHAT THEY ANSWER)



SYSTEM OVERVIEW

Is the system healthy overall?
Any red flags I should investigate?



APPLICATION PERFORMANCE

Are users experiencing slow responses or errors?



INFRASTRUCTURE HEALTH

Are nodes, CPU, memory, or disk under pressure?



KUBERNETES CLUSTER

Are pods restarting? Any failing pods or unhealthy workloads?



BUSINESS METRICS

Is the business healthy?
Are key events happening?

ALERTING (FROM NOISE TO SIGNAL)



REAL PROBLEMS, NOT ANY CHANGE

Alert on symptoms that impact users or reliability.



ACTIONABLE ALERTS

Every alert must have a clear runbook or next step.



RIGHT THRESHOLDS

Set thresholds based on baseline and user impact.



ESCALATE SMARTLY

Different severities, routing, and escalation policies.



MEASURE ALERT QUALITY

Track alert volume, false positives, and mean time to acknowledge.

SLI CONCEPTS



SLI (SERVICE LEVEL INDICATOR)

A quantitative measure of service health from the user perspective.



EXAMPLES OF SLIs

- ✓ Request success rate (%)
- ✓ Request latency (p95 / p99)
- ✓ Availability (uptime %)
- ✓ Throughput (requests/sec)



WHY IT MATTERS

SLIs tie technical metrics to user experience and business outcomes.

EXAMPLE SLI BREAKDOWN

SLI	TARGET	MEASURED BY	WHY IT MATTERS
AVAILABILITY	99.9%	Successful requests / total requests	Users can access the system.
LATENCY (P95)	< 300ms	95th percentile response time	Users get fast responses.
ERROR RATE	< 1%	5xx errors / total requests	Low errors = happy users.
THROUGHPUT	> 100 RPS	Requests per second	System can handle the load.

ALERT THRESHOLD EXAMPLES



CPU USAGE

Warning: > 70% for 5m
Critical: > 90% for 5m



MEMORY USAGE

Warning: > 75% for 5m
Critical: > 90% for 5m



ERROR RATE

Warning: > 1% for 5m
Critical: > 5% for 5m



LATENCY (P95)

Warning: > 300ms for 5m
Critical: > 800ms for 5m



POD RESTARTS

Warning: > 3 in 10m
Critical: > 5 in 10m

WHAT HAPPENS WHEN THINGS FAIL? (WHAT YOU SHOULD SEE)



POD FAILURE

- Pod goes down
- Alert fires
- Traffic shifted
- New pod scheduled
- System recovers



NODE FAILURE

- Node becomes NotReady
- Pods rescheduled
- Alerts notify
- No user impact



DEPENDENCY FAILURE

- Dependency latency up
- Error rate increases
- Circuit breaker opens
- System degrades safely



HIGH TRAFFIC / LOAD

- CPU/memory increase
- HPA scales pods
- Throughput maintained
- Alerts stay green



CONFIG CHANGE ERROR

- Change applied
- Error metrics spike
- Alerts detect
- Rollback triggered

OUTCOME



YOU SEE IT.
YOU UNDERSTAND IT.
YOU FIX IT.
YOU PREVENT IT.

BEST PRACTICES



INSTRUMENT EVERYTHING



USE RED METHOD



DASHBOARDS ANSWER QUESTIONS



KEEP ALERTS ACTIONABLE



SET SLI & SLOs



REVIEW ALERTS REGULARLY



LOG WITH CONTEXT



MONITOR FOR THE USER

DEVSECOPS PIPELINE PROJECT

INTEGRATE SECURITY INTO EVERY STAGE OF DELIVERY.
SHIFT LEFT. AUTOMATE SECURITY. STOP RISKS EARLY.



SECURE CI/CD PIPELINE – END TO END



SECURITY CONTROLS IN THE PIPELINE

- SAST**
Static analysis of source code to find security issues.
- DEPENDENCY SCANNING**
Detect vulnerable open source libraries and packages.
- SECRET SCANNING**
Find hardcoded secrets, keys, tokens, and credentials.
- CONTAINER SCANNING**
Scan container images for known vulnerabilities.
- IAC SCANNING**
Scan Terraform, CloudFormation, Helm charts, and Kubernetes YAML.
- LEAST-PRIVILEGE CREDENTIALS**
Use minimal permissions for users, services, and automation.
- IMAGE POLICIES**
Enforce signed images, allowed registries, and approved bases.
- VULNERABILITY THRESHOLDS**
Define severity thresholds that must be met to continue.
- PIPELINE SECURITY GATES**
Fail the pipeline if security checks do not meet policy.
- SECURITY FINDINGS**
Aggregate and report all security findings with context.
- REMEDiation WORKFLOW**
Track, assign, and verify remediation until the risk is resolved.

SECURITY SCANNING STAGES

SAST	DEPENDENCY SCANNING	SECRET SCANNING	CONTAINER SCANNING	IAC SCANNING
WHAT IT CHECKS:	WHAT IT CHECKS:	WHAT IT CHECKS:	WHAT IT CHECKS:	WHAT IT CHECKS:
- Code quality - Security flaws - Insecure patterns	- Known CVEs - Outdated libs - License risks	- API keys - Tokens - Passwords	- Base image CVEs - OS packages - Misconfigurations	- Misconfigurations - Public exposure - Compliance issues
TOOLS EXAMPLES:	TOOLS EXAMPLES:	TOOLS EXAMPLES:	TOOLS EXAMPLES:	TOOLS EXAMPLES:
- SonarQube - Checkmarx - Semgrep	- OWASP Dependency Check - Snyk - Trivy	- GitGuardian - TruffleHog - GitLeaks	- Trivy - Clair - Grype	- Checkov - tfsec - KICS

PIPELINE SECURITY GATES



LEAST-PRIVILEGE PRINCIPLES

- Use dedicated service accounts with minimal permissions.
- Avoid using long-lived credentials.
- Store secrets securely in Vault or cloud secret managers.
- Rotate credentials automatically.
- Audit all access and changes.
- Enforce MFA for human access.

WHAT HAPPENS WHEN A CRITICAL VULNERABILITY IS DETECTED?

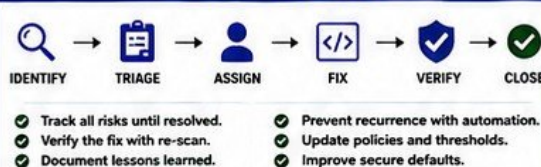
- SCANNER DETECTION**
A scanner finds a CRITICAL vulnerability (e.g., CVSS score ≥ 9.0).
- GATE FAILURE**
The vulnerability exceeds the allowed threshold policy.
- PIPELINE STOPS**
Pipeline fails immediately. Deployment is blocked. No artifact is promoted.
- ALERT & NOTIFY**
Security team and developers are notified via Slack / Email.
- CREATE TICKET**
An issue is created in Jira / GitHub / Azure DevOps for tracking.
- REMEDiate**
Developers fix the issue and commit the changes.
- RE-SCAN & VERIFY**
Scanners run again. Issue is resolved. Policy is met.
- PIPELINE CONTINUES**
Pipeline proceeds. Artifact is deployed to the next stage.

NO CRITICAL VULNERABILITIES MAKE IT TO PRODUCTION.

SECURITY FINDINGS SHOULD INCLUDE

- Vulnerability ID and severity
- Component / file / image details
- Affected version and fixed version
- CVSS score and impact
- Exploitability and references
- Recommended remediation
- Status and owner

REMEDiation WORKFLOW



OPERATIONAL OUTCOMES

- Faster delivery with built-in security.
- Reduced risk in production.
- Consistent, enforceable policies.
- Full visibility into security posture.
- Audit-ready and compliance-friendly.
- Security is everyone's responsibility.

BUILD SECURE. DELIVER CONFIDENTLY. PROTECT USERS. PROTECT BUSINESS.

PRODUCTION INCIDENT CASE STUDY



TURN A FAILURE INTO PORTFOLIO EVIDENCE

EXAMPLE: APPLICATION LATENCY INCREASES IMMEDIATELY AFTER DEPLOYMENT.
THIS DEMONSTRATES TROUBLESHOOTING ABILITY, NOT JUST DEPLOYMENT ABILITY.

INCIDENT SUMMARY



INCIDENT ID

INC-2024-05-18-001



DATE & TIME

MAY 18, 2024
14:32 UTC



DURATION

47 MINUTES



SEVERITY

SEV-2 (HIGH IMPACT)

1 INCIDENT SYMPTOMS

- API response time increased from ~120ms to ~2500ms
- Error rate remained low
- CPU and memory normal
- No deployment failures



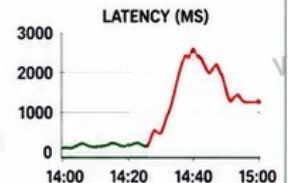
2 INITIAL HYPOTHESIS

- Database slow query
- External dependency slowness
- Cache miss increase
- New code inefficiency



3 METRICS

- p95 latency spiked
- Database connections near max
- Cache hit ratio dropped
- RPS steady



4 LOGS

- Increased DB connection wait time
- Slow query warnings
- Connection pool exhausted



5 INVESTIGATION TIMELINE

14:32 Alert triggered
14:35 Checked metrics & logs
14:40 Identified DB connection pool exhaustion
14:45 Confirmed via tracing
14:50 Root cause identified
15:00 Mitigation applied
15:19 Service recovered



6 ROOT CAUSE

New deployment introduced a connection leak in the application. Connections were not closed properly, exhausting the database connection pool.



7 MITIGATION

- Increased DB connection pool size
- Restarted application pods to release leaked connections
- Enabled connection pool monitoring



8 RECOVERY

- Latency returned to normal after mitigation
- Connection pool stabilized
- No data loss
- Full functionality restored



9 VERIFICATION

- p95 latency back to ~110ms
- Connection pool usage normal
- No new errors
- Monitored for 30 minutes post-recovery



10 PREVENTIVE CHANGES

- Fixed connection leak in code
- Added automated tests for connection handling
- Added connection pool saturation alert
- Implemented circuit breaker for DB calls
- Added runbook for similar incidents



11 LESSONS LEARNED

- Small code changes can cause large impact
- Connection management is critical
- Monitoring should cover saturation, not just errors
- Good alerts + dashboards reduce MTTR
- Runbooks and tests prevent repeat incidents



INCIDENT IMPACT SUMMARY



USERS AFFECTED
~6,200



DOWNTIME
47 MINUTES



BUSINESS IMPACT
DEGRADED
EXPERIENCE



DATA LOSS
NONE



MTTR
47 MINUTES

WHY THIS MATTERS IN YOUR PORTFOLIO

- Shows real-world problem solving
- Demonstrates observability usage
- Proves debugging and analysis skills
- Shows ownership and accountability
- Highlights preventive thinking

RELIABILITY AND FAILURE ENGINEERING PROJECT



DESIGN FAILURE INTO THE PORTFOLIO INTENTIONALLY

A RELIABLE SYSTEM IS PROVEN, NOT ASSUMED.
ENGINEER FAILURES. OBSERVE. RESPOND. IMPROVE.

FOR EVERY TEST DOCUMENT THIS FLOW



TEST SCENARIOS

- Pod failure
- Node failure
- Unhealthy deployment
- Database connectivity loss
- Dependency timeout
- CPU saturation
- Incorrect configuration
- Failed health checks

FAILURE ENGINEERING TEST MATRIX

SCENARIO	FAILURE (WHAT I DID)	DETECTION (HOW I KNEW)	RESPONSE (WHAT TRIGGERED)	RECOVERY (HOW I FIXED IT)	PREVENTION (HOW I WILL PREVENT IT)
POD FAILURE	Deleted a running application pod	Prometheus alert: Pod down High 5xx errors	Alertmanager notification Auto restart by Deployment	New pod created automatically Traffic restored	Liveness probes Resource limits Pod anti-affinity
NODE FAILURE	Terminated a worker node	Node down alert Pod rescheduled events	Cluster autoscaler provisioned replacement node	Pods rescheduled automatically No downtime	Multi-AZ nodes Cluster autoscaler PodDisruptionBudget
UNHEALTHY DEPLOYMENT	Deployed broken image	Readiness probe failed High error rate	Deployment rollout stopped Alert triggered	Rolled back to previous healthy version	Canary deployments Automated tests Pre-deploy checks
DATABASE CONNECTIVITY LOSS	Blocked DB port in security group	DB connection errors Latency spike	Alert triggered Connection retries exhausted	Restored DB access Updated security group	Security group rules Validation tests Runbook update
DEPENDENCY TIMEOUT	Simulated slow external API	High latency APM traces Timeout errors	Circuit breaker opened Alert triggered	Fallback response served Traffic stabilized	Timeout thresholds Circuit breaker Caching
CPU SATURATION	Stress test with CPU load	CPU > 90% Alert triggered Slow response	HPA scaled pods vertically or horizontally	CPU returned to normal Performance stable	Resource limits HPA tuning Performance tests
INCORRECT CONFIGURATION	Deployed wrong config value	Application errors Config validation failed	Alert triggered Rollback started	Correct config deployed Service restored	Config validation GitOps review Automated tests
FAILED HEALTH CHECKS	Broken health endpoint	Health check failed Unhealthy alerts	Kubernetes removed pod from service Alert triggered	Pod restarted Service stabilized	Improved health checks Synthetic monitoring

WHY THIS MATTERS

- ✓ Proves your system can survive real failures
- ✓ Shows observability and alerting capabilities
- ✓ Demonstrates automation and self-healing
- ✓ Prevents the same issues from recurring
- ✓ Employers value problem solvers, not just builders

EVIDENCE TO INCLUDE

- ✓ Test plan and scenarios
- ✓ Screenshots of alerts
- ✓ Metrics before, during, after
- ✓ Logs and traces
- ✓ Timeline of the incident
- ✓ What changed after each test
- ✓ Runbooks and documentation

FAILURE ENGINEERING PRINCIPLES

- ⚠ ASSUME FAILURE
- 🔄 TEST CONTINUOUSLY
- 🔍 DETECT EARLY
- ⚙ RESPOND AUTOMATICALLY
- 📈 IMPROVE ALWAYS

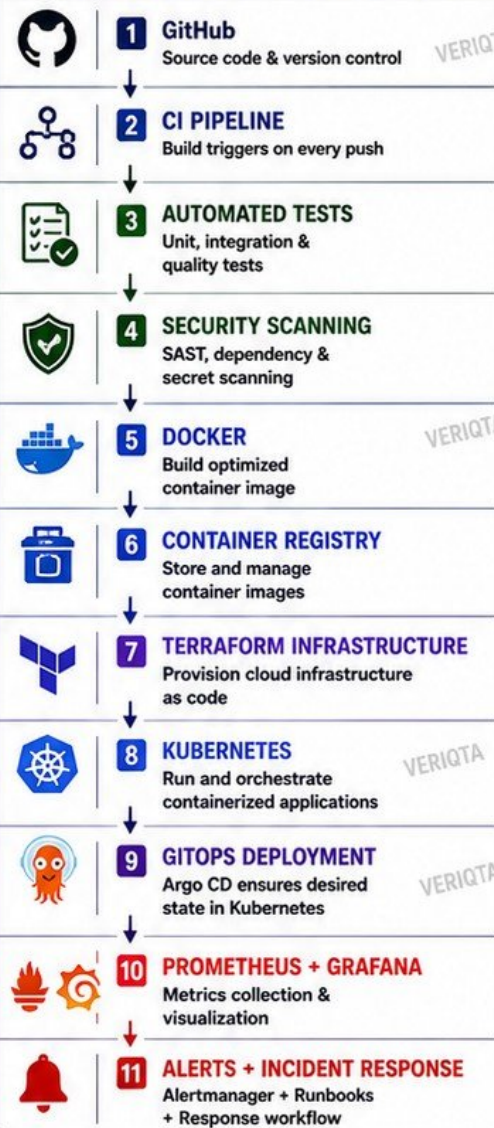
COMPLETE END-TO-END DEVOPS PORTFOLIO PROJECT



COMBINE THE MAJOR SKILLS INTO ONE PRODUCTION-STYLE SYSTEM

DESIGN. AUTOMATE. DEPLOY. OBSERVE. SECURE. RELY.

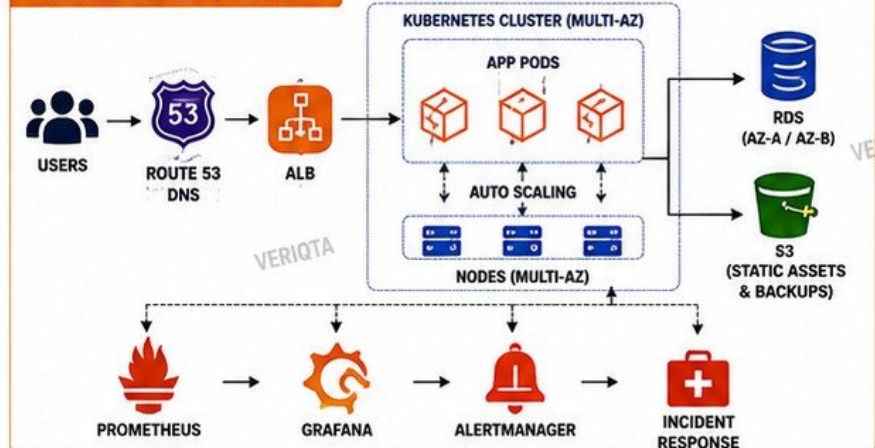
END-TO-END ARCHITECTURE FLOW



WHAT THIS PROJECT DEMONSTRATES



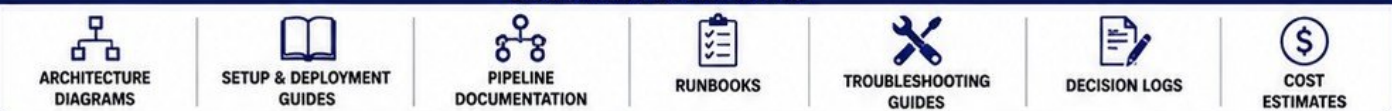
SAMPLE SYSTEM OVERVIEW



WHY THIS BECOMES A STRONG PINNED REPOSITORY

- ✓ Real-world, end-to-end production workflow
- ✓ Includes security, reliability, and observability
- ✓ Includes failure handling, rollback, and recovery
- ✓ Shows depth across the full DevOps lifecycle
- ✓ Demonstrates automation and engineering decisions
- ✓ Well-documented with diagrams and runbooks

DOCUMENT EVERYTHING



PRODUCTION READINESS CHECKLIST

- ✓ Infrastructure as Code
- ✓ Automated CI/CD Pipeline
- ✓ Automated Tests
- ✓ Security Scanning in Pipeline
- ✓ Containerization Best Practices
- ✓ Kubernetes Best Practices
- ✓ GitOps Deployment
- ✓ Observability & Alerting
- ✓ High Availability Architecture
- ✓ Backup & Disaster Recovery
- ✓ Rollback Strategy
- ✓ Incident Response Plan
- ✓ Documentation & Diagrams
- ✓ Troubleshooting Evidence
- ✓ Cost Optimization
- ✓ Everything Reproducible

HOW TO PRESENT PROJECTS TO EMPLOYERS



BE PREPARED TO EXPLAIN YOUR ENGINEERING THINKING, NOT JUST THE TOOLS.

**EMPLOYERS HIRE PROBLEM SOLVERS, NOT TOOL USERS.
SHOW YOUR REASONING. SHOW YOUR IMPACT.**

1 WHAT PROBLEM DID YOU SOLVE?



- Define the real problem clearly.
- Who was affected?
- What was the business impact?
- Why was it important to solve?
- What were the pain points?

2 WHY DID YOU CHOOSE THIS ARCHITECTURE?



- Explain your architectural decisions.
- How does it meet the requirements?
- What principles or patterns did you use?
- Why is this the right fit?
- How does it support the goals?

3 WHAT ALTERNATIVES DID YOU CONSIDER?



- List the other approaches you evaluated.
- Compare pros and cons.
- Why did you not choose them?
- What trade-offs were involved?
- Show you can think beyond one option.

4 WHAT COULD FAIL?



- Identify potential failure points.
- Consider system, network, and dependencies.
- What are the risks and blast radius?
- What assumptions are you making?
- Show that you think about reliability.

5 HOW WOULD YOU DETECT THE FAILURE?



- What metrics, logs, and alerts will help?
- What tools and dashboards are used?
- What are your SLOs or thresholds?
- How quickly will the issue be detected?
- Show your observability strategy.

6 HOW WOULD YOU RECOVER?



- What is your recovery plan?
- How will you restore service?
- Do you have fallback or rollback options?
- How do you minimize downtime?
- Show your incident response readiness.

7 HOW IS THE SYSTEM SECURED?



- How do you protect data and access?
- What security controls are in place?
- How are secrets and credentials managed?
- How do you prevent common threats?
- Show security is designed in, not added.

8 HOW DOES IT SCALE?



- How does it handle more load?
- What scales automatically?
- What are the scaling limits?
- How do you test scalability?
- Show that you plan for growth.

9 WHAT DOES IT COST?



- What are the major cost components?
- How did you optimize costs?
- What is the estimated monthly cost?
- How does it scale cost-effectively?
- Show FinOps awareness.

10 WHAT WOULD YOU CHANGE AT ENTERPRISE SCALE?



- What are the limitations today?
- What would you improve for scale?
- How would you support more teams?
- What automation or abstractions help?
- Show forward-thinking and ownership.

PRESENT ENGINEERING REASONING, NOT JUST TECHNOLOGIES



THINK LIKE AN
ENGINEER



EXPLAIN TRADE-OFFS
AND DECISIONS



SHOW IMPACT,
NOT JUST FEATURES



DEMONSTRATE
PROBLEM SOLVING



OWN THE SYSTEM
END-TO-END



BE CLEAR, CONFIDENT,
AND CONCISE



**EMPLOYERS DON'T WANT PERFECT PROJECTS.
THEY WANT ENGINEERS WHO CAN THINK, DECIDE, AND DELIVER.**



FINAL PORTFOLIO READINESS CHECKLIST



USE THIS CHECKLIST TO ENSURE YOUR DEVOPS PORTFOLIO IS PRODUCTION-READY AND EMPLOYER-READY.

1 REPOSITORY QUALITY

- ☐ CLEAR PROJECT PURPOSE
- ☐ PROFESSIONAL README
- ☐ ARCHITECTURE DIAGRAM
- ☐ CLEAN REPOSITORY STRUCTURE
- ☐ REPRODUCIBLE DEPLOYMENT
- ☐ NO EXPOSED CREDENTIALS



2 ENGINEERING

- ☐ INFRASTRUCTURE AS CODE
- ☐ AUTOMATED CI/CD
- ☐ TESTING
- ☐ SECURITY CONTROLS
- ☐ MONITORING
- ☐ LOGGING
- ☐ HEALTH CHECKS
- ☐ ROLLBACK STRATEGY
- ☐ FAILURE HANDLING



3 PRODUCTION THINKING

- ☐ AVAILABILITY EXPLAINED
- ☐ SCALING EXPLAINED
- ☐ SECURITY DECISIONS EXPLAINED
- ☐ COST CONSIDERATIONS DOCUMENTED
- ☐ TRADE-OFFS DOCUMENTED
- ☐ TROUBLESHOOTING DEMONSTRATED
- ☐ INCIDENT SCENARIO INCLUDED
- ☐ RECOVERY PROCESS TESTED



4 PRESENTATION

- ☐ STRONG PINNED REPOSITORIES
- ☐ GITHUB PROFILE IS ORGANIZED
- ☐ PROJECTS CAN BE EXPLAINED WITHOUT READING THE README
- ☐ SCREENSHOTS SUPPORT TECHNICAL EVIDENCE
- ☐ DIAGRAMS ARE READABLE
- ☐ EVERY MAJOR TECHNOLOGY SERVES A CLEAR PURPOSE



FINAL PORTFOLIO STANDARD

A STRONG
DEVOPS PORTFOLIO
SHOULD PROVE:



I CAN
BUILD IT.



I CAN
AUTOMATE IT.



I CAN
SECURE IT.



I CAN
OBSERVE IT.



I CAN
TROUBLESHOOT IT.



I CAN
RECOVER IT
WHEN IT FAILS.



AND I CAN
EXPLAIN WHY
I DESIGNED IT
THIS WAY.



YOUR PORTFOLIO IS YOUR ENGINEERING STORY.
MAKE IT REAL. MAKE IT CLEAR. MAKE IT IMPACTFUL.

